Machine Learning in Python – Final Project

A Comparative Analysis of Machine Learning Algorithms for Predicting Student Dropout and Academic Success

Yehudis Schnaidman

Part 1: Data Exploration

The dataset I chose for researching the performance of machine learning algorithms is the Predict Students' Dropout and Academic Success dataset from the UCI Machine Learning Repository. The dataset contains 4,424 student records with 36 input features and one target variable. The features describe different aspects of each student's background, enrollment information, academic performance, and socioeconomic conditions. Examples of recorded information include age at enrollment, gender, course information, admission grade, semester grades, completed curricular units, tuition payment status, scholarship status, and parents' education and occupations.

The dataset contains both numerical and categorical features. While all features are represented using numerical values in the dataset, many of these values represent categories rather than measurements. For example, gender, course, marital status, and scholarship status are categorical variables encoded as numbers. Numerical features include continuous measurements such as admission grade, previous qualification grade, semester grades, unemployment rate, inflation rate, and GDP.

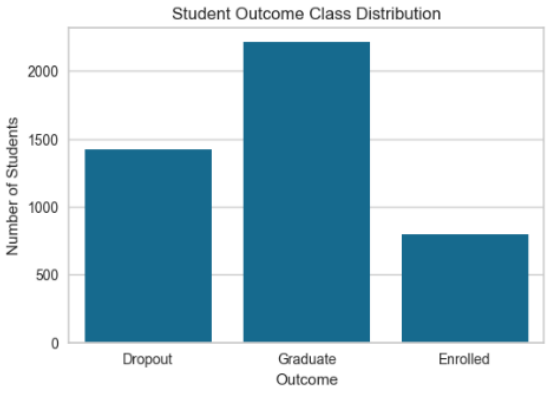
The target variable is Target, which represents the student's academic outcome. It contains three classes: Dropout, Enrolled, and Graduate.

This is a classification problem because the goal is to predict which predefined category a student belongs to. The model is not predicting a continuous value but instead assigning each student to one of three possible classes.

This is a machine learning problem because the relationship between student characteristics and academic outcomes is complex. Instead of manually defining rules to predict success or dropout, machine learning algorithms can learn patterns from historical student data and use those patterns to make predictions on new students.

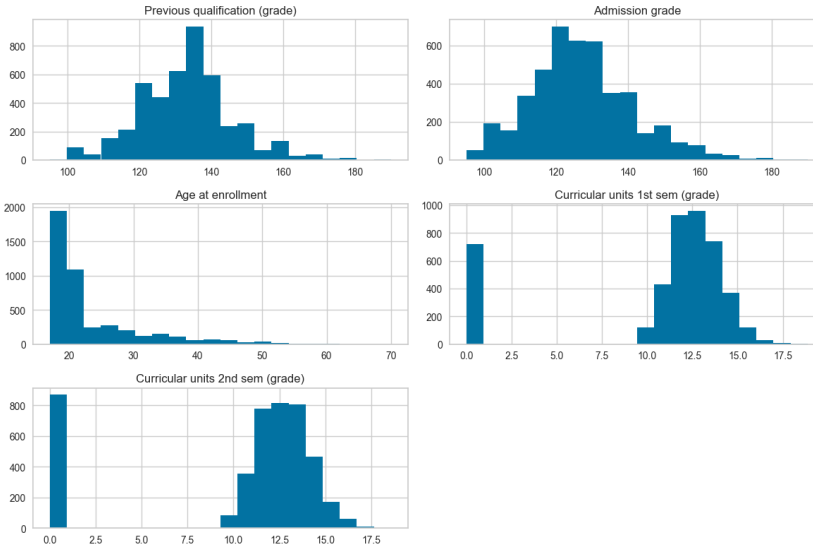
The dataset contains 2,209 students classified as Graduate, 1,421 students classified as Dropout, and 794 students classified as Enrolled.

The class distribution was visualized using a bar chart to examine the balance between the three target classes. The dataset is not perfectly balanced because the Graduate class contains more examples than the Enrolled class.

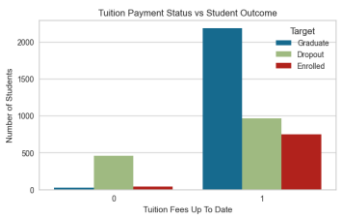


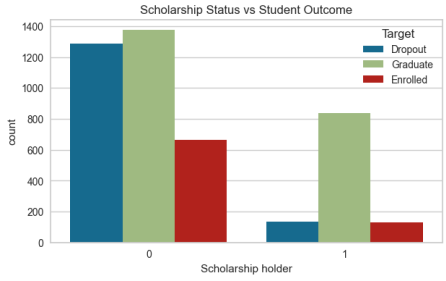
The dataset was checked for missing values. No missing values were found, so no imputation was required.

Feature distributions were examined using histograms of important numerical variables. The previous qualification grades and admission grades were mostly concentrated around the 120–140 range. The age distribution showed that most students enrolled between approximately 18 and 25 years old. The first and second semester curricular unit grades showed that many students either received a grade of zero or achieved grades between approximately 10 and 16.



The Tuition Payment Status versus Student Outcome chart showed that students who were not up to date with tuition payments had a higher proportion of dropouts. The Scholarship Status versus Student Outcome chart showed that scholarship holders had a higher proportion of graduates.





The correlation heatmap was used to examine relationships between numerical features. Strong positive relationships were observed between academic performance features, especially between first and second semester grades and approved curricular units. These relationships make sense because students who perform well in one academic area often perform well in other academic areas. Economic features such as unemployment rate and GDP showed relationships with each other because they represent related economic conditions during the students' enrollment periods.

Before training the machine learning models, the data was prepared by separating the features from the target variable. Since machine learning algorithms require numerical class labels, the target classes were converted into numerical values using LabelEncoder. The categorical features were transformed using OneHotEncoder because their numerical values represented categories rather than quantities. Numerical features were standardized using StandardScaler, which transforms the features so they have a mean of 0 and a standard deviation of 1. This prevents features with larger ranges from having a greater influence on models that are sensitive to feature scale. Finally, the dataset was divided into training and testing sets using an 80/20 split with random_state=42 to ensure reproducibility.

Part 2: Machine Learning

Model 1: Logistic Regression

The first machine learning model implemented was Logistic Regression. The model was trained using the prepared dataset with the default Logistic Regression settings.

The Logistic Regression model achieved a training accuracy of 0.69 and a testing accuracy of 0.66. The similar training and testing accuracy values indicate that the model did not significantly overfit the training data. The model required approximately 0.97 seconds to train and 0.002 seconds to make predictions on the testing dataset, demonstrating that Logistic Regression is computationally efficient.

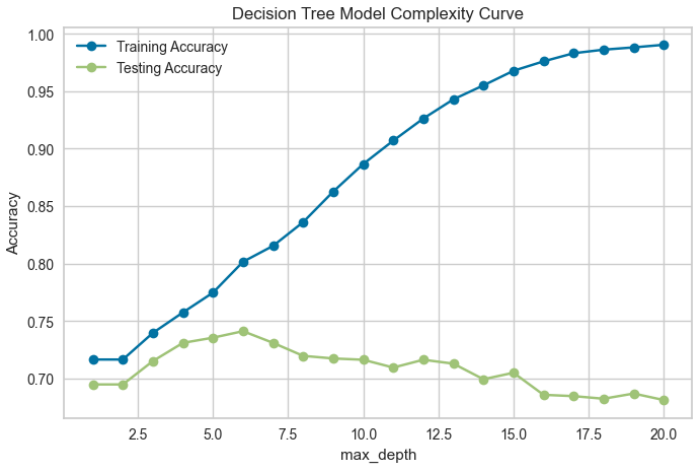
The model achieved moderate performance due to the complexity of predicting three student outcome classes. The confusion matrix and classification report showed that the model identified Graduate students well but struggled with the Enrolled class, with most Enrolled students incorrectly classified as other outcomes.

Model 2: Decision Tree

The second machine learning model implemented was the Decision Tree model. The model was first trained using the default Decision Tree settings and achieved a training accuracy of 1.00 and a testing accuracy of 0.68. The large difference between training and testing performance indicated that the default tree was overfitting the training data.

To reduce overfitting, different values of max_depth were tested and a model complexity curve was created to compare training and testing accuracy. The max_depth parameter controls the maximum number of levels in the decision tree. Increasing max_depth allows the model to learn more complex patterns, but excessive depth can increase variance and cause overfitting. Smaller depths may cause underfitting due to high bias.

The complexity curve showed that increasing max_depth improved training accuracy but eventually reduced testing accuracy, indicating that deeper trees were fitting the training data too closely and generalizing poorly. The best balance between training and testing performance was found at max_depth = 6. The final Decision Tree model was retrained using this value and achieved a training accuracy of 0.80 and a testing accuracy of 0.74. The model required 0.03 seconds for training and 0.003 seconds for querying the testing data, demonstrating efficient performance.



Model 3: AutoML & Hyperparameter Tuning

The third machine learning experiment used AutoML (Automated Machine Learning) with the PyCaret classification library. The AutoML search was configured with a time budget of 120 seconds, a random seed of 42 to ensure reproducibility, and Accuracy as the optimization metric.

AutoML is a process that automatically searches through multiple machine learning algorithms and their hyperparameter combinations to find the model that performs best on a given dataset. Instead of manually selecting a model and testing different parameter values, AutoML evaluates many possible models and configurations automatically.

Hyperparameters are settings that control how a machine learning model learns. Examples include the number of trees in an ensemble model, the maximum depth of trees, and the minimum number of samples needed for a split. Automating this search can often produce better results than manual tuning because it can test many more combinations and compare multiple algorithms, reducing the chance of choosing a suboptimal model.

The best model selected by PyCaret was the Extra Trees Classifier. The hyperparameters chosen by the AutoML search included n_estimators = 100, while max_depth, max_leaf_nodes, and max_samples were all set to None, allowing the model to grow without these restrictions.

The AutoML model achieved a training accuracy of 0.77 and a testing accuracy of 0.77. The total AutoML search time was 24.7 seconds, and the query time on the testing data was 0.35 seconds.

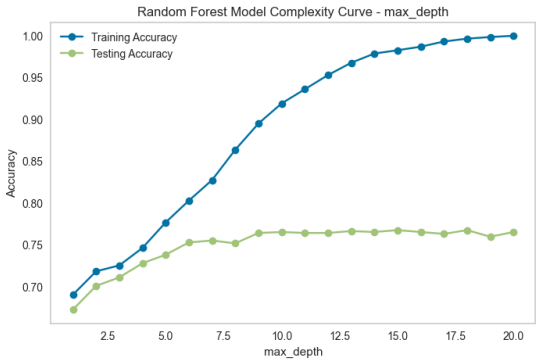
AutoML found a slightly better model, achieving 0.77 testing accuracy compared to 0.74 for the tuned Decision Tree and 0.75 for the tuned Random Forest, because it automatically

searched through multiple algorithms and hyperparameter combinations and identified the Extra Trees Classifier, which achieved a slightly better balance between bias and variance.

Model 4: Random Forest

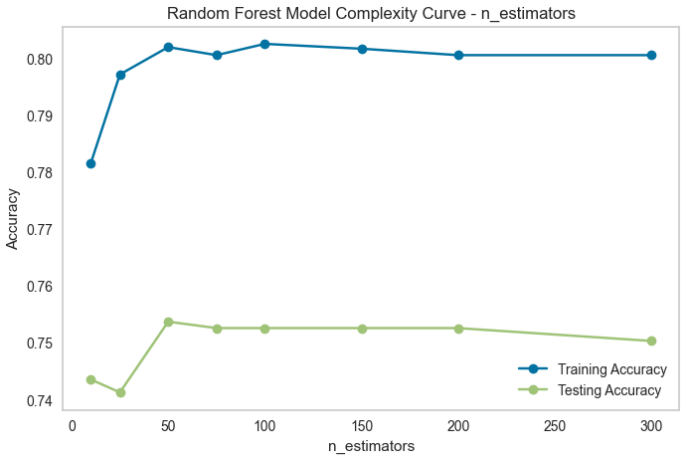
The fourth machine learning model implemented was the Random Forest model.

A model complexity curve was created to determine the optimal value for max_depth. max_depth represents the maximum number of levels allowed in each decision tree in the forest. The selected value for max_depth was 6 because it provided a good balance between bias and variance. At lower depths, the model had higher bias because the trees were too simple to capture important patterns in the data.



After depth 6, training accuracy continued increasing significantly, while testing accuracy remained almost unchanged. This indicates that deeper trees were learning more details from the training data without improving generalization, increasing the model's variance. Therefore, max_depth=6 was selected because it achieved strong performance while reducing the risk of overfitting.

A second model complexity curve was created to determine the optimal value for n_estimators while keeping max_depth=6. The selected value for n_estimators was 50 because it achieved the best testing accuracy while maintaining efficient training time. Increasing the number of trees beyond 50 did not provide a significant improvement in testing performance. Since additional trees increase computational cost, 50 trees provided a good balance between accuracy and efficiency.



n_estimators is the number of decision trees in the random forest. Increasing it usually improves performance because more trees reduce variance and make the model more stable. However, more trees also increase training and prediction time, and after a certain point the improvement becomes very small.

Different trees can be created from the same training set using bagging. Bagging creates different training datasets by randomly sampling the original data with replacement, so

each tree learns from a slightly different dataset. Random forests add another level of randomness by randomly selecting a subset of features at each split. This creates diverse trees, and combining their predictions improves accuracy and reduces overfitting.

Using the selected parameters, max_depth=6 and n_estimators=50, the final Random Forest model achieved a training accuracy of 0.80 and a testing accuracy of 0.75. The training time was 0.41 seconds, and the query time was 0.014 seconds.

Model 5: Neural Network (PyTorch)

The fifth machine learning model implemented was a feedforward Neural Network using PyTorch. The network was created by subclassing nn.Module and was designed to classify the dataset into three target classes: Dropout, Enrolled, and Graduate. The final architecture consisted of two hidden layers with 64 and 32 neurons, followed by an output layer with 3 neurons representing the three classes

Different network architectures, learning rates, and numbers of epochs were tested to determine the best-performing model. The final model used two hidden layers with 64 and 32 neurons, which meets the minimum requirement of two hidden layers. A smaller model with one hidden layer ([32]) achieved a lower testing accuracy of 74.9%, showing that the additional hidden layer helped the model learn more complex patterns. A deeper model with three hidden layers ([128, 64, 32]) achieved slightly higher training accuracy but a slightly lower testing accuracy (77.3% compared to 77.4%). This suggests that the additional complexity did not improve the model's ability to generalize to unseen data. Therefore, the two-hidden-layer model was selected because it provided the best testing performance while maintaining a simpler architecture.

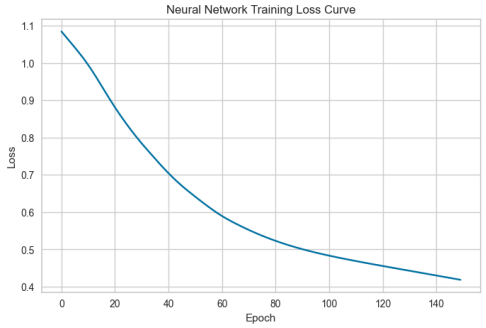
The hidden layers used ReLU activation functions. ReLU introduces non-linearity into the network, allowing the model to learn more complex relationships within the data rather than only learning simple linear patterns.

The best results were achieved using the Adam optimizer, a learning rate of 0.001, and 150 epochs. Adam was selected because it adjusts the learning rate during training, allowing the model to converge efficiently. After testing different epoch values, 150 epochs was chosen because increasing the number of epochs did not significantly improve testing accuracy and only increased training time.

The model was trained using an explicit training loop. At each epoch, optimizer.zero_grad() was used to reset the gradients from the previous training step. The model then performed a forward pass by sending the training data through the neural network to generate

predictions. The CrossEntropyLoss function was used because this was a multi-class classification problem with three possible output classes. It measured the difference between the predicted probabilities and the true labels. Next, loss.backward() calculated the gradients needed for learning, and optimizer.step() updated the model weights to improve future predictions.

The training-loss curve shows that the loss decreased throughout training, meaning the model was learning and converging. The model was able to learn the patterns in the data without showing major signs of underfitting or overfitting.



The final model achieved a training accuracy of approximately 0.84 and a testing accuracy of approximately 0.77. The model took approximately 0.99 seconds to train and approximately 0.04 seconds to make predictions. The model was evaluated using model.eval() inside a torch.inference_mode() block, which switches the model to testing mode and prevents unnecessary gradient calculations, allowing predictions to run more efficiently.

Part 3: Comparing and Contrasting Performance

	Training Accuracy	Testing Accuracy	Training Time	Query Time
Logistic Regression	.69	.66	.97	.002
Decision Tree	.80	.74	.027	.002
AutoML	.77	.771	24.7	.35
Random Forest	.80	.75	.49	.017
Neural Network	.84	.774	.99	.04
Best	Neural network	Neural network	Decision Tree	Logistic Regression / Decision Tree
Worst	Logistic Regression	Logistic Regression	AutoML	AutoML

The Neural Network achieved the best overall performance, but its improvement over the other models was very small. It achieved the highest testing accuracy, only slightly outperforming AutoML and Random Forest. This suggests that while the neural network was able to learn useful patterns from the dataset, the classical models were also able to perform well on this problem.

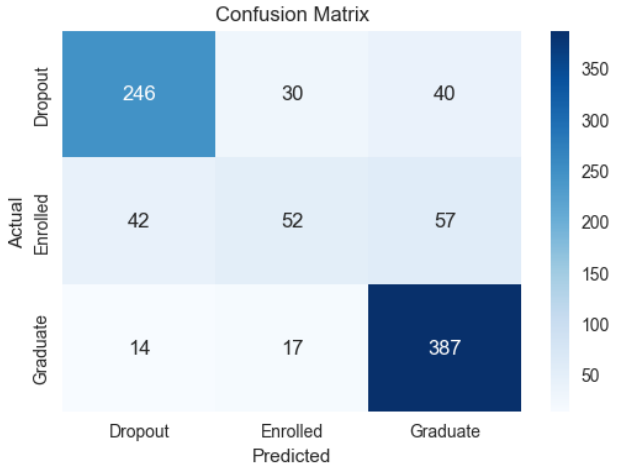
AutoML had the longest training time because it automatically tested multiple model types and hyperparameter combinations in order to find the best-performing model. In contrast, the Decision Tree trained much faster because it is a simpler model that only needs to learn a set of decision rules from the data.

Unlike many small tabular datasets where neural networks may not provide an advantage, this dataset contained enough complexity and relationships between student features for the neural network to perform well. However, the small difference between the neural network and classical models suggests that the dataset did not require a very complex model to achieve strong results.

Logistic Regression performed the worst in terms of accuracy because it is a simpler model that assumes mostly linear relationships between features and the target variable. Since student outcomes are influenced by many interacting academic, financial, and demographic factors, a more flexible model was better able to capture these patterns.

The Neural Network required more training and query time compared to most classical models because it contains multiple layers of neurons and must perform many calculations during training. The model learns by repeatedly adjusting its weights through backpropagation, which requires more computation than simpler algorithms such as Logistic Regression or Decision Trees.

The Neural Network confusion matrix shows that the model performed best on the Graduate class. The most common confusion occurred with the Enrolled class, where 57 students who were actually Enrolled were predicted as Graduate. Enrolled students were also often confused with Dropout students.



This likely happens because Enrolled students share characteristics with both groups. For example, students with strong academic performance may look similar to graduates, while students with weaker performance or financial difficulties may resemble dropouts.

The classification report shows that the Graduate class had the best performance with a precision of 0.80, recall of 0.93, and F1-score of 0.86. The Dropout class also performed well with a precision of 0.81, recall of 0.78, and F1-score of 0.80. The Enrolled class had the weakest performance with a precision of 0.80, recall of 0.34, and F1-score of 0.42. The low recall means that many actual Enrolled students were incorrectly classified as another class. The weaker performance on the Enrolled class may also be partially caused by class imbalance, since Enrolled had fewer examples than Graduate and Dropout.

GitHub Repo:

https://github.com/yehudis25/student-academic-success-ml

Predict Students' Dropout and Academic Success Dataset:

https://archive.ics.uci.edu/dataset/697/predict+students+dropout+and+academic+success